CRAFT Demo Script

A simple guide for presenting seven parts of CRAFT. Every part follows the same order:

1. **What it is**: one sentence
2. **Open it**: who logs in and where to click
3. **Say**: what to read out loud
4. **Fields**: what is on the screen and where it is saved
5. **How it connects**: the files, from the screen down to the database
6. **Key code**: one short piece of code that does the main job

Before you start

Demo accounts (password for all: `Campus123`)

Role	Account ID	Opens on
Admin	`ADMIN-0001`	Administrator Dashboard
Custodian	`EMP-1001`	Custodian Dashboard
Student	`2023-104857`	Borrower Dashboard
Faculty	`EMP-2001`	Borrower Dashboard

Folder names used in this guide

Short name	Full folder	What it holds
`WebApp/`	`ASI.Basecode.WebApp/`	Screens (Views) and Controllers
`Services/`	`ASI.Basecode.Services/`	Business rules
`Data/`	`ASI.Basecode.Data/`	Database access and table models

How every screen works

> Screen (View) → Controller → Service → Repository → Database

- The **Controller** receives the click or form.
- The **Service** applies the rules.
- The **Repository** reads and writes the database.
- All data is stored in the SQL Server database **`AllianceDB`**.

Where things are plugged together

What	File
Database address	`WebApp/appsettings.json` → `AllianceConnection`
Database is switched on	`WebApp/Startup.cs`
Each Service and Repository is registered	`WebApp/Startup.DI.cs`
Each table is mapped	`Data/AllianceDbContext.cs`
Demo data is loaded at startup	`Services/Services/DatabaseSeeder.cs`

1. Equipment Details

What it is: The list of all equipment. Staff can add, edit and delete items, upload a photo and print a QR label.

Open it: Log in as Admin → sidebar **Equipment Administration** → click an item name.
(Custodian: sidebar **Equipment Management**.)

Say: "This is our equipment catalogue. Each item has a record with its photo and stock count, and we can print a QR label for it."

Fields

Field	What it means	Saved in
Equipment Name	Name of the item	`equipment.equipment_name`
Asset Code	Unique tag, e.g. LPT-2024-018	`equipment.asset_code`
Category	Type of item, e.g. Laptops	`equipment_categories`
Location	Where it is kept	`locations`
Availability Status	Available or Under Maintenance	`equipment.operational_status_id`
Condition Classification	Good, Minor Damage, Damaged, Missing Parts, Lost	`equipment.current_condition_id`
Total Quantity	Number of units (1 to 10,000)	`equipment_inventory_balances.total_quantity`
Description	Notes, setup, accessories (up to 500 characters)	`equipment.description`
Equipment Photo	JPG, PNG, WEBP or GIF, up to 5 MB	`equipment.image_path`
Active	Can it be reserved?	`equipment.is_active`
Available / Reserved / Released	Live stock counts	`equipment_inventory_balances`

The list page also shows totals (Equipment Records, Total Units, Available, Reserved, Released) and filters (search, category, location, condition, status).

How it connects

1. **Screen:** `WebApp/Views/Equipment/` (Index, Details, Create, Edit, Label)
2. **Controller:** `WebApp/Controllers/EquipmentController.cs`: list, add, edit, delete, details, QR label
3. **Service:** `Services/Services/EquipmentService.cs`: checks the asset code is unique, saves the item and photo
4. **Repository:** `Data/Repositories/EquipmentRepository.cs`: reads and saves equipment rows
5. **Database:** tables `equipment` and `equipment_inventory_balances`. Photos are saved in `WebApp/wwwroot/uploads/equipment/`.

Key code: how "Available" is calculated

`WebApp/Views/Equipment/Details.cshtml`

```
var available = Math.Max(0, balance.TotalQuantity - balance.ReservedQuantity
    - balance.ReleasedQuantity - balance.AwaitingInspectionQuantity
    - balance.MaintenanceQuantity - balance.UnreconciledQuantity);
```

Available = total units minus everything that is reserved, borrowed, being inspected or under repair.

2. Equipment Specifications

What it is: The details that describe an item. CRAFT has no separate "specifications" screen; the specifications are the fields on the equipment record, and the **Description** box holds the technical details.

Open it: Log in as Admin → **Equipment Administration** → **Edit** on any item → show the **Description** box.

Say: "Each item's specifications are stored on its record: name, asset code, category, condition and photo. Technical details like RAM or accessories go in the Description."

Fields

Specification	Entered in	Saved in
Model / name	Equipment Name	`equipment.equipment_name`
Unique tag	Asset Code	`equipment.asset_code`
Type	Category	`equipment_categories.category_name`
Physical state	Condition Classification	`equipment.current_condition_id`
Can it be used?	Availability Status, Active	`equipment.operational_status_id`, `equipment.is_active`
Home location	Location	`equipment.location_id`
Quantity	Total Quantity	`equipment_inventory_balances.total_quantity`
Technical details	Description	`equipment.description`
Picture	Equipment Photo	`equipment.image_path`

How it connects

Same path as Part 1: **EquipmentController** → **EquipmentService** → **EquipmentRepository** → table **`equipment`**.
The form is `WebApp/Views/Equipment/_EquipmentForm.cshtml`. New categories are added in **System Settings** (Part 6).

Key code: the equipment record

`Data/Models/Equipment.cs`

```
public string AssetCode { get; set; }
public string EquipmentName { get; set; }
public string Description { get; set; }
public long EquipmentCategoryId { get; set; }
public long CurrentConditionId { get; set; }
public long OperationalStatusId { get; set; }
public bool IsActive { get; set; }
```

Each property is one column in the `equipment` table.

> **If asked:** "Structured specs per category, such as a separate specifications table, would be a future improvement."

3. Borrowing Rules

What it is: The limits on borrowing: how long, how many units, and how far ahead. The Admin sets them; the system checks them every time someone reserves.

Open it:

1. Admin → **System Settings** → **Borrowing Policy** card (where the rules are set)
2. Student → **Equipment** → open an item → **Borrowing Rules** card (where the rules are shown)
3. Click **Reserve Equipment** and enter a quantity above the limit to show the error (where the rules are checked)

Say: "The admin sets the rules once. Every reservation is checked against them, so a student can't request more units or more days than allowed."

> `BorrowerController` only shows the borrower's dashboard. The rules live in **SettingsService** (where they're saved) and **ReservationService** (where they're checked).

Fields: the rules

Rule	Current value	Saved in `borrowing_policy_versions`
Max Borrow Days	7 days	`max_borrow_duration_days`
Faculty Max Borrow Days	21 days	`faculty_max_borrow_days`
Max Units per Reservation	3	`max_units_per_borrower`
Max Open Requests per Borrower	3	`max_active_borrowing_units`
Reservation Lead Time	1 day ahead	`reservation_lead_time_days`
Max Advance Booking	30 days ahead	`max_advance_booking_days`
Overdue Grace	60 minutes	`overdue_grace_period_minutes`
Block if overdue	On	`block_reservation_when_overdue`

Checked when a reservation is submitted, in this order:

1. Quantity: at least 1, within the max units, and no more than what's available
2. Date: not in the past, at least the lead time ahead, within the advance booking limit
3. Account: verified (unverified accounts can't reserve)
4. Length: within Max Borrow Days (faculty: Faculty Max Borrow Days)
5. Open requests: below the limit
6. Overdue items: blocked if the borrower has any
7. Both agreement boxes ticked

How it connects

1. **Screens:** `WebApp/Views/Settings/Index.cshtml` (set), `WebApp/Views/Reservations/EquipmentDetails.cshtml` (show), `WebApp/Views/Reservations/New.cshtml` (reserve)
2. **Controllers:** `SettingsController.cs` (save rules), `ReservationsController.cs` (submit reservation)
3. **Services:** `Services/Services/SettingsService.cs` (saves the rules); `Services/Services/ReservationService.cs`: `GetActivePolicy` reads them and `Create` checks them
4. **Repository:** `Data/Repositories/SettingsRepository.cs`
5. **Database:** table `borrowing_policy_versions` (the row marked current is the active one)

Key code: checking the rules

`Services/Services/ReservationService.cs` (method `Create`, shortened)

```
if (form.Quantity > policy.MaxUnitsPerBorrower)
    return (false, ErrorMessages.ReservationQuantityAboveMax, ...);

if (activeCount >= policy.MaxActiveTransactions)
    return (false, ErrorMessages.ReservationActiveLimitReached, ...);

if (policy.BlockReservationWhenOverdue && this.HasOverdueBorrowing(...))
    return (false, ErrorMessages.ReservationOverdueBlock, ...);
```

If any rule fails, the reservation is stopped and the student sees an error message.

4. Custodian Dashboard

What it is: The custodian's to-do list for the day: what to approve, release, collect and inspect.

Open it: Log in as Custodian `EMP-1001`. It opens automatically (sidebar **Dashboard**).

Say: "This is the custodian's work for today. The boxes at the top show what needs attention, the task list shows each job, and the alerts warn about low stock or items under maintenance."

Fields

Section	Field	What it shows
At a glance	Pending Reservations	Requests waiting for approval
	Releases Today	Approved items to hand out today
	Active Borrowings	Items currently borrowed
	Returns Today	Items due back today
	Overdue Equipment	Items past their due time
	Awaiting Inspection	Returned items not yet checked
Task List	Task, Type, When, Open button	One row per job
Inventory Alerts	Item name + Low Stock / Under Maintenance	Items at or below the Low Stock Threshold

A custodian sees jobs assigned to them plus jobs not yet assigned. An admin sees everything.

How it connects

1. **Screen:** `WebApp/Views/Custodian/Dashboard.cshtml`
2. **Controller:** `WebApp/Controllers/CustodianController.cs`, method `Dashboard`
3. **Service:** `Services/Services/DashboardService.cs`, method `GetCustodianDashboard` (counts the jobs)
4. **Data access:** the service reads the database directly through `AllianceDbContext` (no separate repository)
5. **Database:** tables `reservations`, `borrowings`, `equipment`, `equipment_inventory_balances`

Key code: counting the jobs

`Services/Services/DashboardService.cs`

```
var pending = reservations.Where(r => r.Status.Code == "PENDING").ToList();
var overdue = openBorrowings
    .Where(b => BorrowingRules.IsOverdue(b.Status.Code, b.DueAtUtc, cutoff)).ToList();
```

"Overdue" means past the due time plus the grace period. That one definition lives in `Services/Manager/BorrowingRules.cs`, so every screen agrees.

5. Operational Dashboard

What it is: The Admin's overview of the whole system. It is read-only.

Open it: Log in as Admin `ADMIN-0001`. It opens automatically (sidebar **Dashboard**). The screen is titled **Administrator Dashboard**.

Say: "This is the big picture: how many users and items we have, what is borrowed or overdue, and a log of recent activity."

> This is **MonitoringController**, not OperatorController. `OperatorController` is an old developer tool ("Switch Operator") with no menu link.

Fields

Section	Fields
At a glance	Total Users, Equipment Records, Available Units, Reserved Units, Borrowed Units, Overdue Transactions, Damaged / Lost
Reservation statistics	Total, Pending, Approved, Ready for Release, Completed, Rejected, Cancelled
Account verification	Pending, Approved, Rejected, Median time to decision
Custodian workload	Each custodian's Pending and Approved count, with a Reassign option
Recent system activity	When, Actor, Action, Entity, Reference, Details

The same section of the menu also has **Reservations & Transactions**, **Reports** and **Audit Logs**, with CSV export.

How it connects

1. **Screen:** `WebApp/Views/Monitoring/Dashboard.cshtml`
2. **Controller:** `WebApp/Controllers/MonitoringController.cs`, method `Dashboard`
3. **Service:** `Services/Services/MonitoringService.cs`, method `GetDashboard`
4. **Repository:** `Data/Repositories/MonitoringRepository.cs`: counts reservations, borrowings and equipment, and reads the audit log
5. **Database:** tables `users`, `equipment`, `reservations`, `borrowings`, `custodian_assignments`, `audit_logs`

Key code: loading the dashboard

`WebApp/Controllers/MonitoringController.cs`

```
public IActionResult Dashboard()
{
    this.ViewBag.Workload = this._assignments.GetWorkload();
    this.ViewBag.VerificationStats = this._communication.GetVerificationStats();
    return this.View(this._service.GetDashboard());
}
```

The controller collects three sets of numbers (workload, verification, totals) and sends them to the screen.

6. System Settings

What it is: The Admin's control page for the borrowing rules, equipment categories and locations.

Open it: Log in as Admin → sidebar **System Settings**.

Say: "Changing a rule here never overwrites the old one. It publishes a new version, so we keep a full history. Categories and locations are managed here too."

Fields

Borrowing Policy

Field	Allowed values
Max Borrow Days	1 – 365
Max Units per Reservation	1 – 100
Max Open Requests per Borrower	1 – 100
Faculty Max Borrow Days	1 – 365
Reservation Lead Time (days)	0 – 30
Max Advance Booking (days)	1 – 365
Overdue Grace (min)	0 – 1,440
Low Stock Threshold	0 – 10,000
Block reservations when borrower has overdue items	on / off

Button: **Publish New Policy Version**. It is rejected if nothing changed.

Policy History: Version, Effective From, Current, Max Days, Units / Reservation, Open Requests

Equipment Categories: Code (e.g. `LAPTOP`), Name. A category can't be deleted while equipment uses it.

Locations: Code (e.g. `LIB-2F`), Name, Pickup point, Active

How it connects

1. **Screen:** `WebApp/Views/Settings/Index.cshtml`
2. **Controller:** `WebApp/Controllers/SettingsController.cs`: save policy, add/delete category, add/delete location
3. **Service:** `Services/Services/SettingsService.cs`: checks the values and creates the new version
4. **Repository:** `Data/Repositories/SettingsRepository.cs`
5. **Database:** tables `borrowing_policy_versions`, `equipment_categories`, `locations`. Every change is also recorded in `audit_logs`.

Key code: publishing a new version

`Services/Services/SettingsService.cs` (method `SavePolicy`, shortened)

```
// versioning: never update — create a new row, mark previous current=false
var previous = this._repository.GetCurrentPolicy();
previous.IsCurrent = false;
nextVersion = "v" + (n + 0.1m).ToString("0.0");   // v1.0 → v1.1
```

The old rules stay in the database; the new row becomes the current one.

> Not the same as **Profile & Settings** (avatar menu), which is each user's personal contact details and notifications.

7. System Rules Configuration

What it is: All the settings that control how CRAFT behaves. They are kept in four places.

Say: "Most rules are set by the Admin on screen. The timing of the automatic checks is in a configuration file, a few security limits are fixed in the code, and the Admin also decides which custodian handles which requests."

#	Where	What it controls	How to change it
A	**System Settings** screen	Borrowing rules (Part 3 and 6)	Admin, on screen
B	`WebApp/appsettings.json` → `ReservationLifecycle`	Automatic checks: expiring old requests, overdue marking, reminders	Edit the file, restart the app
C	Code	Login lockout, reset link expiry, photo size, items that can't be released	Developer changes the code
D	**Custodian Assignments** screen	Which custodian handles which borrower, location or category	Admin, on screen

A. Borrowing rules

See Part 3 and Part 6. Saved in `borrowing_policy_versions`.

B. Automatic checks (config file)

Setting	Value	Meaning
Enabled	true	Automatic checks are on
AutoCancelHours	24	Requests not approved within 24 hours expire
DueReminderHours	24	Borrowers get a reminder 24 hours before the due time
IntervalMinutes	5	The check runs every 5 minutes

Key code: `WebApp/appsettings.json`

```
"ReservationLifecycle": {
  "Enabled": true,
  "AutoCancelHours": 24,
  "DueReminderHours": 24,
  "IntervalMinutes": 5
}
```

Runs in: `WebApp/Mvc/ReservationLifecycleWorker.cs` (the timer) → `Services/Services/ReservationLifecycleService.cs` (does the work).

C. Fixed in the code

Rule	Value	File
Login lockout	5 wrong passwords → locked 15 minutes	`Services/Services/AuthService.cs`
Password reset link	Expires after 24 hours	`Services/Services/AuthService.cs`
Equipment photo	Max 5 MB; JPG, PNG, WEBP, GIF	`WebApp/Controllers/EquipmentController.cs`
Can't be released	Damaged, Lost, Under Maintenance	`Services/Services/CustodianService.cs`
What "overdue" means	Past due time plus the grace period	`Services/Manager/BorrowingRules.cs`

D. Custodian Assignments

Open it: Admin → sidebar **Custodian Assignments**.
Fields: one section each for **by borrower**, **by location** and **by category**. Pick the target, pick a custodian, click **Assign**. The table shows the target, the custodian and the start date, with a **Remove** button.

How it connects: `WebApp/Controllers/CustodianAssignmentsController.cs` → `Services/Services/CustodianAssignmentService.cs` → table `custodian_assignments`

Quick reference

#	Screen	Open at	Controller	Service	Main table
1	Equipment Details	`/Equipment`	EquipmentController	EquipmentService	`equipment`
2	Equipment Specifications	`/Equipment/Edit/{id}`	EquipmentController	EquipmentService	`equipment`
3	Borrowing Rules	`/Settings`, `/Reservations/New/{id}`	SettingsController, ReservationsController	SettingsService, ReservationService	`borrowing_policy_versions`
4	Custodian Dashboard	`/Custodian/Dashboard`	CustodianController	DashboardService	`reservations`, `borrowings`
5	Operational Dashboard	`/Monitoring/Dashboard`	MonitoringController	MonitoringService	`reservations`, `borrowings`, `audit_logs`
6	System Settings	`/Settings`	SettingsController	SettingsService	`borrowing_policy_versions`, `equipment_categories`, `locations`
7	System Rules Configuration	`/Settings`, `/CustodianAssignments`, `appsettings.json`	SettingsController, CustodianAssignmentsController	SettingsService, ReservationLifecycleService	`borrowing_policy_versions`, `custodian_assignments`

Database: SQL Server, database `AllianceDB`, address in `WebApp/appsettings.json` (`AllianceConnection`).